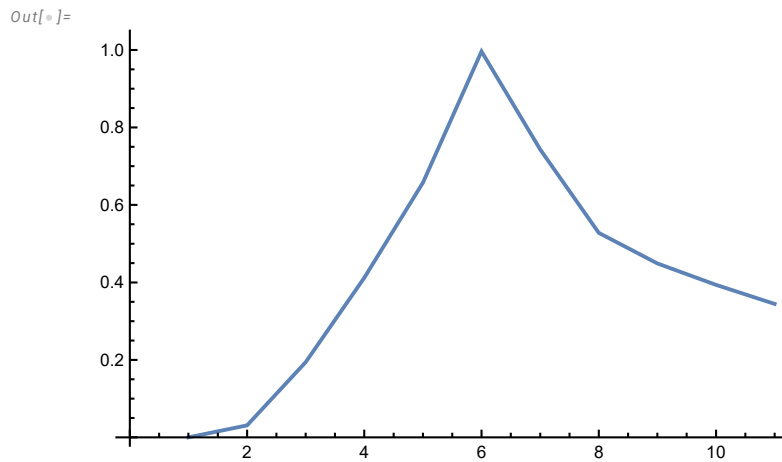


```
In[*]:= (* Random phase JONSWAP spectrum with modal frequency omegam*)
```

```
In[*]:= Table[Om[i], {i, -5, 5}]
```

```
Out[*]:= {0.5, 0.6, 0.7, 0.8, 0.9, 1., 1.1, 1.2, 1.3, 1.4, 1.5}
```

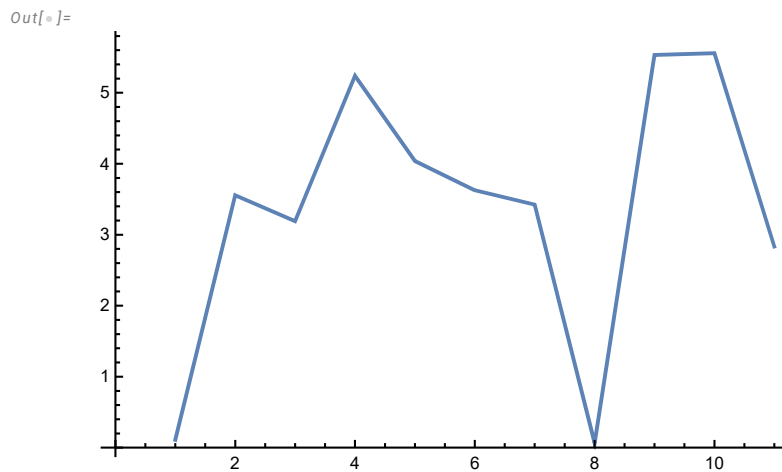
```
In[*]:= ListLinePlot[Table[A[i], {i, -5, 5}]]
```



```
In[*]:= Table[A[i], {i, -5, 5}]
```

```
Out[*]:= {0.000275988, 0.0310087, 0.194104, 0.411984, 0.657557,  
0.996291, 0.743132, 0.527962, 0.449042, 0.393818, 0.344679}
```

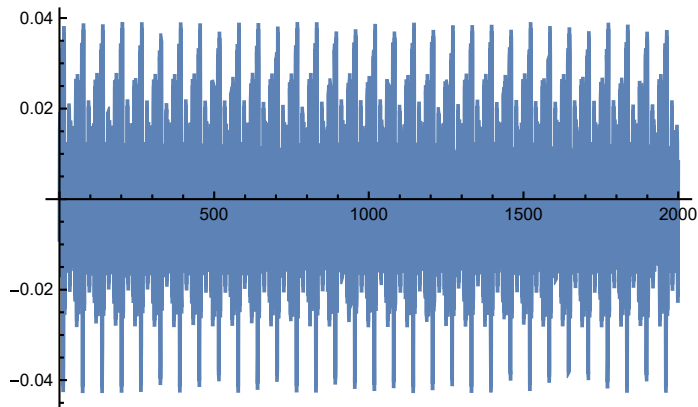
```
In[*]:= ListLinePlot[Table[CC[i], {i, -5, 5}]]
```



```
In[*]:= gg[t_] := Sum[A[i] × Cos[Om[i] t + CC[i]], {i, -5, 5}]
```

```
In[*]:= Plot[-eps gg'[t], {t, 0, 2000}]
```

```
Out[*]:=
```



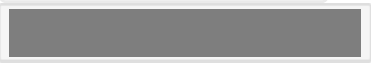
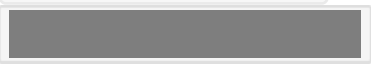
```
In[*]:= eqz1ent := z1'[t] + z1[t] + eps lam (z1'[t] - z2'[t]) + eps k (z1[t] - z2[t]) == -eps gg'[t]
```

```
In[*]:= eqz2ent := z2'[t] + z2[t] + eps lam (z2'[t] - z1'[t]) + eps k (z2[t] - z1[t]) +  
eps lama (z2'[t] - vz'[t]) + eps ka (z2[t] - vz[t])^3 == -eps gg'[t]
```

```
In[*]:= eqz3ent :=  
eps vz'[t] - eps lama (z2'[t] - vz'[t]) - eps ka (z2[t] - vz[t])^3 == -eps^2 gg'[t]
```

```
In[*]:= solnzent = NDSolve[{eqz1ent, eqz2ent, eqz3ent, z1[0] == 0, z1'[0] == 0,  
z2[0] == 0, z2'[0] == 0, vz[0] == 0, vz'[0] == 0}, {z1, z2, vz}, {t, 0, 2000}]
```

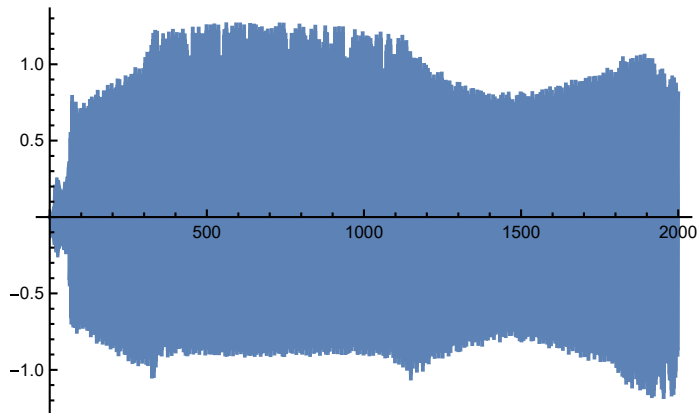
```
Out[*]:=
```

```
{ {z1 → InterpolatingFunction[  
+  Domain: {{0., 2.00 × 10³}}  
Output: scalar  
  
z2 → InterpolatingFunction[  
+  Domain: {{0., 2.00 × 10³}}  
Output: scalar  
  
vz → InterpolatingFunction[  
+  Domain: {{0., 2.00 × 10³}}  
Output: scalar  
 } }
```

```
In[*]:= (*k=1; ka=14/3; lam=0.2; lama=0.2, eps=0.01 *)
```

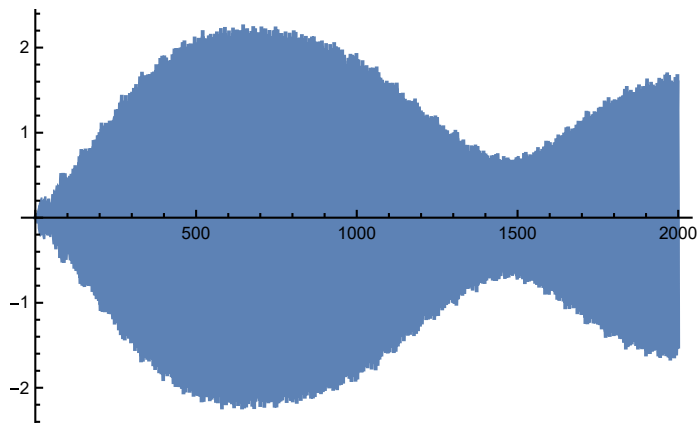
```
In[*]:= plotwz2 = Plot[Evaluate[ {z2[t] - vz[t]} /. solnzent], {t, 0, 2000}, PlotRange -> All]
```

Out[*]=



```
In[*]:= Plotuz = Plot[Evaluate[ {z2[t] + eps vz[t]} /. solnzent], {t, 0, 2000}, PlotRange -> All]
```

Out[*]=



```
In[*]:= wz1[t_] := Evaluate[z1[t] - z2[t] /. solnzent]
```

```
In[*]:= wz2[t_] := Evaluate[z2[t] - vz[t] /. solnzent]
```

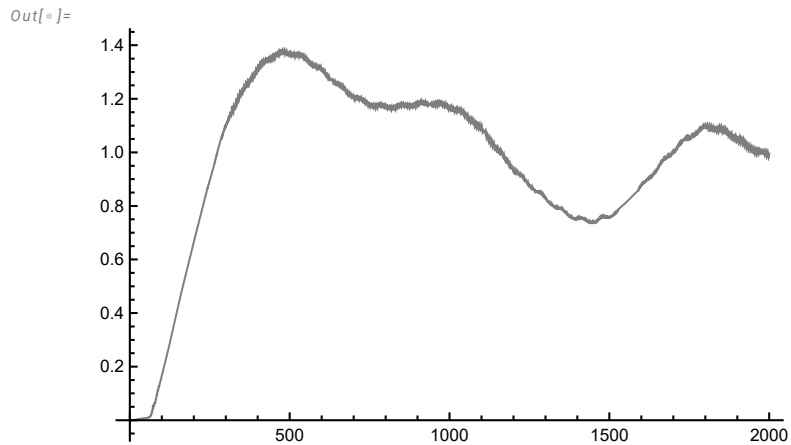
```
In[*]:= uz[t_] := Evaluate[z2[t] + eps vz[t] /. solnzent]
```

```
In[*]:= Phiz1[t_] := wz1'[t] + I wz1[t]
```

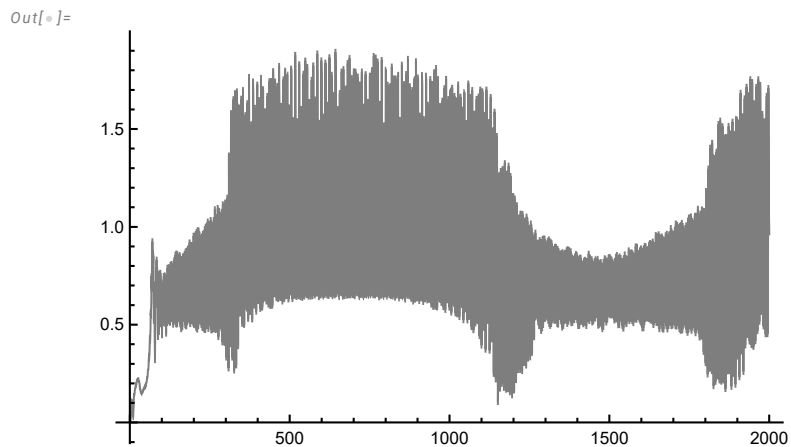
```
In[*]:= Phiz2[t_] := wz2'[t] + I wz2[t]
```

```
In[*]:= Phiz3[t_] := uz'[t] + I uz[t]
```

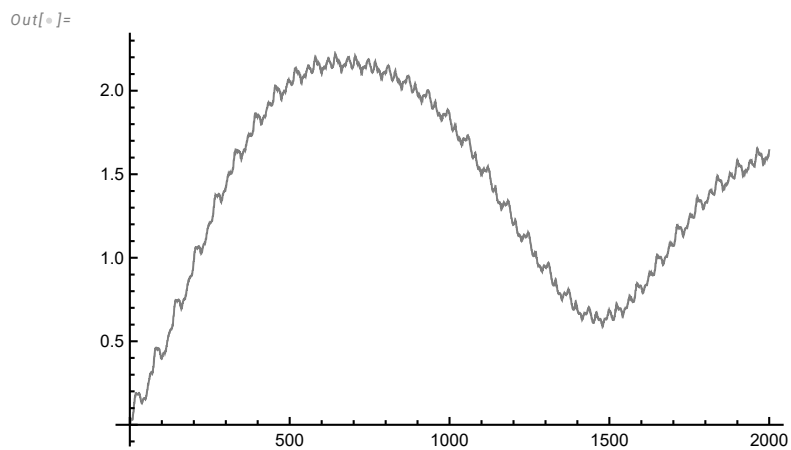
```
In[ ]:= pExactPhiz1 =  
Plot[Abs[Phiz1[t]], {t, 0, 2000}, PlotRange -> All, PlotStyle -> {Thickness[0.001], Gray}]
```



```
In[ ]:= pExactPhiz2 =  
Plot[Abs[Phiz2[t]], {t, 0, 2000}, PlotRange -> All, PlotStyle -> {Thickness[0.001], Gray}]
```



```
In[ ]:= pExactPhiz3 =  
Plot[Abs[Phiz3[t]], {t, 0, 2000}, PlotRange -> All, PlotStyle -> {Thickness[0.001], Gray}]
```

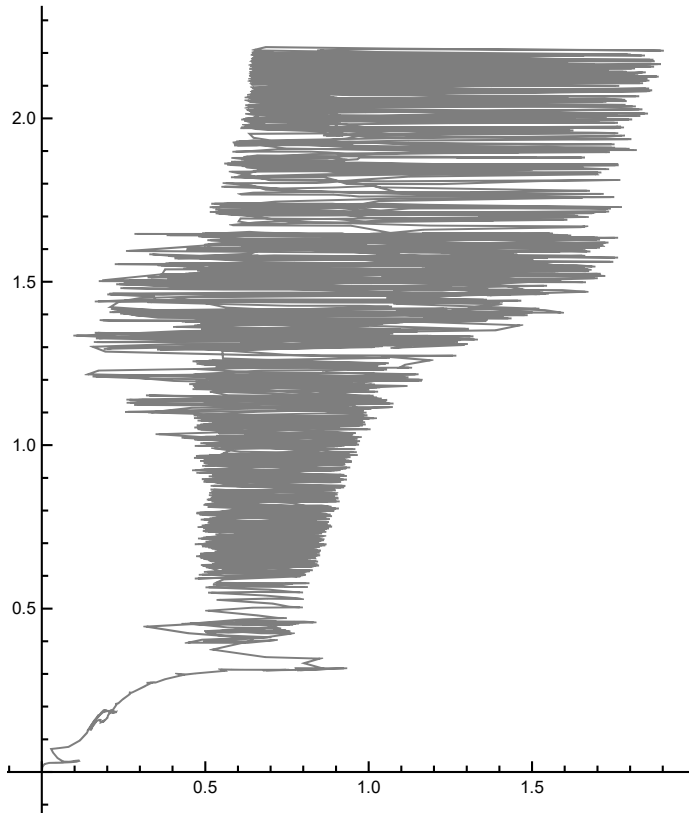


```
In[*]:= ExactPhizProj = ParametricPlot[Evaluate[ {intPz2[t], intPz3[t]} /. solnzent],
      {t, 0, 2000}, PlotStyle -> {Thickness[0.001], Gray}, PlotRange -> All]
```

... InterpolatingFunction: Input value {0.0408163} lies outside the range of data in the interpolating function. Extrapolation will be used.

... InterpolatingFunction: Input value {0.0408163} lies outside the range of data in the interpolating function. Extrapolation will be used.

Out[*]:=

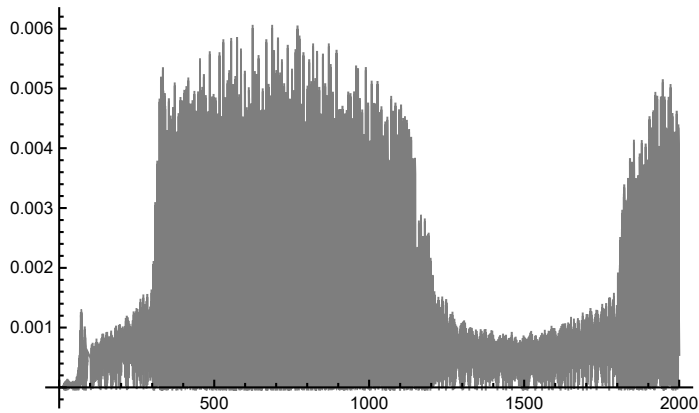


```
In[*]:= (*****Total power dissipated in the system*****)
```

```
In[*]:= (* Nonlinear system *)
```

```
In[*]:= pztotnonl = Plot[Evaluate[eps lama (z2'[t] - vz'[t])^2 /. solnzent],
  {t, 0, 2000}, PlotRange -> All, PlotStyle -> {Thickness[0.001], Gray}]
```

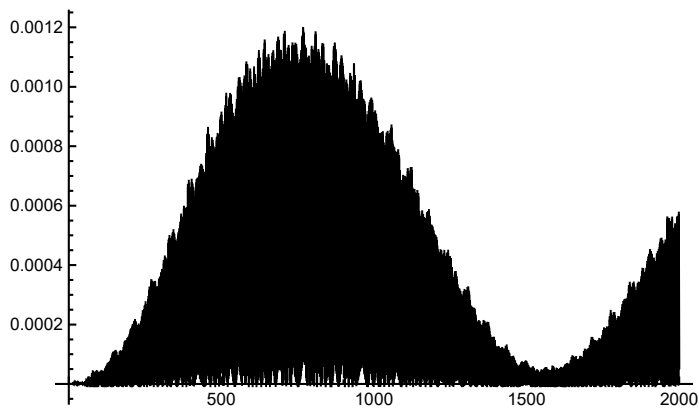
Out[*]=



```
In[*]:= (* Linear system *)
```

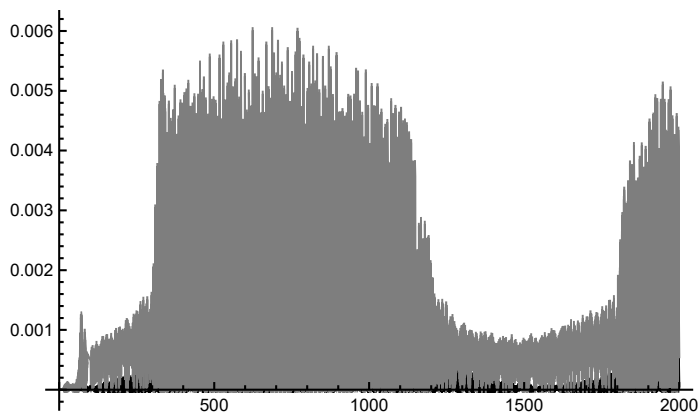
```
In[*]:= pztotlin = Plot[Evaluate[eps lama (z2'[t] - vz'[t])^2 /. solnzent],
  {t, 0, 2000}, PlotRange -> All, PlotStyle -> {Thickness[0.001], Black}]
```

Out[*]=



```
In[*]:= Show[pztotlin, pztotnonl]
```

Out[*]=



```

In[*]:= (*****Slow Flow Analysis and
          comparison to Exact Solution*****)

In[*]:= eqP1 := P1'[t] - eps k I P1[t] - eps (lama / 2) P2[t] +
          (3 / 8) eps ka I Abs[P2[t]]^2 P2[t] + (eps lam / 2) P1[t] == 0

In[*]:= eqP2 :=
          P2'[t] + (I / (2 (1 + eps))) (P2[t] - P3[t]) + eps (k I / 2) P1[t] + ((1 + eps) lama) / 2 P2[t] -
          (3 / 8) (1 + eps) ka I Abs[P2[t]]^2 P2[t] + (eps lam / 2) P3[t] == 0

In[*]:= eqP3 :=
          P3'[t] + (I eps / (2 (1 + eps))) (P3[t] - P2[t]) + I eps (k / 2) P1[t] + (eps lam / 2) P3[t] ==
          (1 + eps) eps 0.9962909987148387 (1 / 2) Exp[I 3.626268087926144]

In[*]:= solnP =
          NDSolve[ {eqP1, eqP2, eqP3, P1[0] == 0, P2[0] == 0, P3[0] == 0}, {P1, P2, P3}, {t, 0, 2000}]

```

Out[*]=

```

{ {P1 → InterpolatingFunction[
  {
     Domain: {{0., 2.00 × 103}}
    Output: scalar
  },
  P2 → InterpolatingFunction[
    {
       Domain: {{0., 2.00 × 103}}
      Output: scalar
    },
    P3 → InterpolatingFunction[
      {
         Domain: {{0., 2.00 × 103}}
        Output: scalar
      }
    ]
  }
}

```

```
In[*]:= Show[PhiProj, ExactPhizProj]
```

Out[*]=

